

CS336 Assignment 4 (data): Filtering Language Modeling Data

第四章的核心任务是构建一个处理 Web 数据的 Pipeline。从原始的 HTML 网页上把原始数据清洗, 得到高质量的结果作为训练集训练语言模型。

主要步骤:

1. **提取 (Extraction)**: HTML 转文本。
2. **过滤 (Filtering)**: 去除有害内容、PII(个人信息)、低质量内容。
3. **去重 (Deduplication)**: 删掉重复信息。

数据来源: Common Crawl(CC) 一个公益网页爬虫平台

三种关键文件格式:

1. **WARC (Web ARChive)**: 最原始数据, 包含 HTTP 请求头 + 原始 HTML。
2. **WAT (Web Archive Transformation)**: 元数据 (Metadata), JSON 格式, 如链接、标题。
3. **WET (Web Extracted Text)**: 官方提取的纯文本。

12.1. Web 网页文本提取

- 目标: 从 raw HTML (bytes) 中提取纯净文本。
- 工具库: Resiliparse(讲义推荐, 适合 CC)
- 主要思路:
 1. 输入是网页内容 str
 2. 先检测是什么编码格式 (UTF-8, GBK, ...)
 3. 利用 Resiliparse 进行 HTML 解析、提取纯文本

Extension 12.1 BeautifulSoup、Trafilatura、Resiliparse 三种网页解析文本工具的比较

BeautifulSoup 适合小规模爬虫, 需要人工编写提取文本的规则。Trafilatura 是目前 LLM 构建数据集里最流行、好用的方式之一, 无需自己写规则, 可以自动得到好的网页文本数据。Resiliparse 优势在于处理超大规模的数据。

- **BeautifulSoup**: 适用于**小规模、定向爬虫**。例如: 你需要从特定的 10 个技术博客抓取文章, 且这些博客结构固定, 你可以写死规则以获得 100% 的提取准确度。

12.1	Web 网页文本提取	157
12.2	语种识别 (Language identification)	158
12.3	个人隐私处理	159
12.4	MinHash	161
12.4.1	为什么要使用 MinHash 算法 (它是用来解决什么问题的)	161
12.4.2	MinHash 的数学原理	161
12.4.3	MinHash 算法流程	161
12.4.4	MinHash 签名矩阵	163
12.5	精确去重与语义去重	165
12.5.1	精确去重	165
12.5.2	语义去重	167

- **Trafilatura**: 适用于**构建通用预训练语料库 (Pre-training Corpus)**。例如: 从任意 URL 列表中提取正文, 不关心具体网站结构, 追求在百万级网页上的平均高质量表现。它是目前 RedPajama 等数据集构建的首选工具之一。
- **Resiliparse**: 适用于**超大规模清洗流水线 (如处理 CommonCrawl)**。当你需要处理 PB 级数据, 且主要瓶颈在于 CPU 解析 HTML 的时间和内存时, 使用 Resiliparse 替换默认的解析器, 或者用来做初步的 ETL 清洗。

12.2. 语种识别 (Language identification)

- 目标: LLM 训练通常集中在特定语言 (如英语), 混合语言数据可能不仅无用还会干扰模型, 语种识别就是为了去除这种混合的语言数据。
- 技术方案: 使用分类器对文本进行打分 (fastText)。
- 主要思路:
 1. 输入文本 -> 输出 ‘(Language Label, Confidence Score)’。
 2. 设定阈值 (Threshold) 过滤非目标语言文档。

Extension 12.2 fastText 技术原理

fastText 是一个**基于线性模型的浅层神经网络**的模型, 其核心优势在于在保持与深度学习模型相当精度的同时, 推理速度比深度神经网络快几个数量级 (通常在 CPU 上即可达到每秒百万词的处理速度)。

1. 子词 (Subword) N-gram 特征提取

- 不同于传统的 Word2Vec 将每个单词视为原子单位, fastText 将单词拆解为字符级别的 N-gram。
- 例如: 单词 “apple” (n=3) 会被表示为 ‘<ap, app, ppl, ple, le>’。
- 优势: 这使得模型能够处理**未登录词 (OOV)**, 并捕捉词根、词缀等形态学信息 (这对处理噪声很大的网页文本至关重要)。

2. 特征平均与映射 (Hidden Layer Averaging)

- 模型将输入文本中所有 N-gram 的向量进行**平均**, 得到整个文档的向量表示。
- 这是一个简单的线性叠加过程, 计算开销极低, 不需要复杂的矩阵乘法或注意力机制计算。

3. 层次 Softmax (Hierarchical Softmax)

- 在输出层进行分类预测时, fastText 不使用标准的 Softmax (计算量随类别数线性增长 $O(N)$)。
- 它构建了一棵**霍夫曼树 (Huffman Tree)**, 将标签预测转化为树上

的路径搜索问题, 计算复杂度降为对数级 $O(\log N)$ 。这使得它能在几秒钟内处理数十万个类别的分类任务

具体示例: 在 LLM 数据清洗中, 最典型的应用是**语种识别 (Language identification)**。

输入数据 (Input Document): "Hello world! 今天天气真不错。Voici un exemple."

处理流程:

1. Tokenization: 拆解为字符 N-gram 序列。
2. Vectorization: 查找 Embedding 表并取平均值。
3. Inference: 模型计算属于各个语言标签的概率。

输出结果 (Output Prediction):

- '(English, 0.6)': 预测为英语, 概率为 0.6。
- '(Chinese, 0.3)': 预测为中文, 概率为 0.3。
- '(French, 0.1)': 预测为法语, 概率为 0.1。

Extension 12.3 N-gram

N-gram(N 元语法) 是指在一段文本序列中, 由连续出现的 N 个基本单元 (Token, 如字符、单词或字节) 组成的滑动窗口片段。

N 值	名称	提取出的 N-grams 集合	逻辑说明
N=1	Unigram (一元)	[我, 是, 一, 头, 猪]	仅关注单个字, 完全丢失上下文顺序信息。
N=2	Bigram (二元)	[我是, 是一, 一头, 头猪]	捕捉相邻两个字的搭配, 能识别"我是"、"一头"等词汇。
N=3	Trigram (三元)	[我是一, 是一头, 一头猪]	捕捉更长的语义依赖,"一头猪"被完整识别。

12.3. 个人隐私处理

- 背景: 防止模型在生成时泄露真实用户的隐私 (训练数据记忆化问题)。
- 处理策略:**掩码 (Masking)**, 即将敏感信息替换为特殊占位符。
- 实现思路: 使用正则表达式匹配敏感信息, 并将其替换为特殊占位符。

Keynote 12.1

简单的正则替换可能会有**误报 (False Positives)**或**漏报 (False Negatives)**, 会对下游模型产生什么影响?

先不讨论漏报误报, 即使正确把所有的关于个人隐私的信息包括 *ip,email,phone number* 全都按照设想转换成了 ‘`|||IP_ADDRESS|||`’ ‘`|||EMAIL_ADDRESS|||`’ ‘`|||PHONE_NUMBER|||`’, 那也会产生这三个替换的结果被过拟合。

漏报的话可能就使得模型学习到别人的隐私信息, 产生**安全风险**; 误报的话就使得原本不是隐私的有价值的信息被替换, **降低模型的性能**。

针对漏报, 要加强一下正则之前的**前期处理**, 把格式搞得标准一些, 不要因为一些全角、半角, 中文等问题影响处理。

针对误报, 可以增加一些**上下文的信息**, 如果一个“IP 地址”前面出现了“Version”, “v”, “Section”等词, 或者后面跟了“release”, 那么可能就是软件版本号, 而不是 IP 地址, 产生了误报。

12.4. MinHash

参考资料 12.1

<https://www.cnblogs.com/sddai/p/6110704.html>

12.4.1. 为什么要使用 MinHash 算法 (它是用来解决什么问题的)

在很多任务 (例如搜索引擎的网页去重、推荐系统) 中, 我们关心的是**两个集合有多相似**, 典型例子:

- 两篇文章是否相似?
- 两个网页是否重复?
- 两段代码是否抄袭?

一个非常经典的相似度指标是:

Jaccard 相似度:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

两个集合的**交集占并集的比例**, 简单理解就是俩集合重合的也就是相同的内容占总内容的比例。问题在于:

- **集合很大** (几十万、几百万 token) **数据量极大** (上亿文档) 会相当占存储
- 直接算交并集会非常**消耗计算量, 时间很漫长**。

MinHash 的解决方案: 它将巨大的集合 (高维向量) 映射为一个**固定长度的短向量 (签名/signature)**, 并保证: **两个集合签名的相似度, 近似等于它们原始集合的 Jaccard 相似度**。

12.4.2. MinHash 的数学原理

核心思想: 如果两个集合原本就很相似, 或者说重复的元素很多, 那么经过某种随机排列的方式之后, 在这个新顺序中出现的第一个元素 (实际为哈希函数映射后的**最小哈希值**) 也很可能相同。反之, 如果两个集合原本就没啥共同元素, 那么经过某种排列后新顺序下各自的第一个元素也大概率不同。因此, 我们可以以这种方式来间接判断两个集合的 Jaccard 相似度。用数学化的语言来表示: 定义 $h(S)$ 为集合 S 在经过随机排列后, 第一个出现的元素, 其中 $h()$ 表示某个哈希函数。

MinHash 定理指出:

$$P(h(A) = h(B)) \approx J(A, B)$$

即两个集合经随机排列后, 它们最小哈希值相等的概率, 近似等于它们的 Jaccard 相似度。

12.4.3. MinHash 算法流程

之前我们说了, 我们希望用随机排列的手段获取新顺序下的最小哈希值; 但如果要像 random shuffle 这种形式的随机排列的话是很耗时的。而使用哈希函数计算模拟随机排序的方法则更加省时。

Random Shuffle 和 Hash 的比较

e.g. 想象一下实际场景：

- **全集元素 (N)**：词汇表可能有 **1,000** 个词。
- **签名长度 (K)**：我们需要 **100** 个哈希值来做签名。

方案 A：使用 Random Shuffle (随机排序)

1. 需要生成 100 个随机排列。
2. 每个排列是对 1000 个数字进行洗牌。这是巨大的内存和时间开销。
3. **最致命的**：如果要算出 S_1 的签名, 你需要遍历这 100 个长度为 1000 的排列序列, 去寻找 S_1 中存在的词。

- **复杂度**： $O(K \times N)$ 。 $100 \times 1000 = 10$ 万次操作才能算出一个文档的签名。

方案 B：使用哈希函数 (MinHash 的做法)

1. 我们不需要真的把 1000 行重新物理排序。
2. 我们只需要定义 100 个简单的公式 (例如 $h(x) = (3x + 7) \bmod N$)。
3. 对于文档 S_1 , 假设它只包含 50 个词 (也就是稀疏向量中只有 50 个 1)。
4. 我们只需要把这 50 个词的行号, 带入到 100 个公式里算一下, 剩下的 950 个元素和文档 S_1 没关系, 因此不用管。

- **复杂度**： $O(K \times L)$ (L 是文档的实际词数)。
- $100 \times 50 = 5000$ 次操作。

原始数据特征矩阵

我们先将原始数据给矩阵数学表示一下

单词	文档 1	文档 2	文档 3
单词 1	1	0	0
单词 2	1	0	0
...			
单词 n	0	0	1

- **定义**：一个布尔矩阵 (只有 0 和 1)。
- **行**：代表**全集元素** (例如：全部文档所有的 100 万个单词)。
- **列**：代表**集合/文档** (例如：你要处理的 600 个文档)。
- **数值**：
 - 1：表示该文档包含该单词。
 - 0：表示该文档不包含该单词。

它的痛点：

1. **极度稀疏 (Sparse)**: 因为一篇文档可能只有 500 个词, 但词表有 100 万个词。所以这一列里有 99.95% 都是 0。
2. **超级巨大**: 100 万行 $\times$ 600 列, 内存根本存不下, 通常只能用稀疏矩阵存储格式 (如链表) 来存。

在实际工程上, 我们往往不会去单独存储原始数据的特征矩阵, 因为太大了, 而是会直接读取文档转换成签名矩阵。

12.4.4. MinHash 签名矩阵

这是 MinHash 算法计算后的结果, 也是我们真正用来做相似度计算的东西。

- **定义**: 一个整数矩阵。
- **行 (Rows)**: 代表**哈希函数** (即我们前面说的 K , 通常取 100~200)。
 - **这里的行不再是单词了, 而是第几个哈希函数。**
- **列 (Columns)**: 代表**集合/文档** (这一点没变, 列依然对应原来的文档)。
- **数值**: 存储的是该文档在该哈希函数下的**最小哈希值**。

它的特点 (优势):

1. **高度稠密 (Dense)**: 里面填满了整数, 没有那么多无意义的 0。
2. **非常小巧**: 行数从 100 万变成了 K (代表我们哈希函数的数量, 也就是 K 种随机排列)。

原始矩阵 $\rightarrow$ 签名矩阵

假设我们有 K 个哈希函数 $h_1, h_2, \dots, h_k$ 我们要构建签名矩阵 M , 其中 $M(i, c)$ 表示第 i 个哈希函数对第 c 个文档算出的签名值。

哈希函数	文档 1	文档 2	文档 3
h_1	1	5	6
h_2	1	1	6
$\dots$			
h_k	4	4	6

算法伪代码逻辑:

1. **初始化**: 把签名矩阵 M 的所有格子都填上无穷大 (∞)。
2. **遍历原始数据**:

我们一行一行地读取原始的特征矩阵 (或者读文档中的词)。

假设当前读到了 **行 r (代表单词 w)** :

- 先计算这一行的 K 个哈希值:

$$val_1 = h_1(r), \quad val_2 = h_2(r), \quad \dots, \quad val_k = h_k(r)$$

- 然后看哪些文档里有这个词 (哪些列是 1):
- 如果文档 c 有这个单词 (特征矩阵中 $(r, c) = 1$):
 - 我们就尝试更新签名矩阵的第 c 列。
 - 规则:

$$M(i, c) = \min(M(i, c), val_i)$$

- 我之前记录的最小值是 ∞ 或者别的数, 现在新来了一个单词, 它的哈希值是 val_i , 如果它更小, 我就更新它。

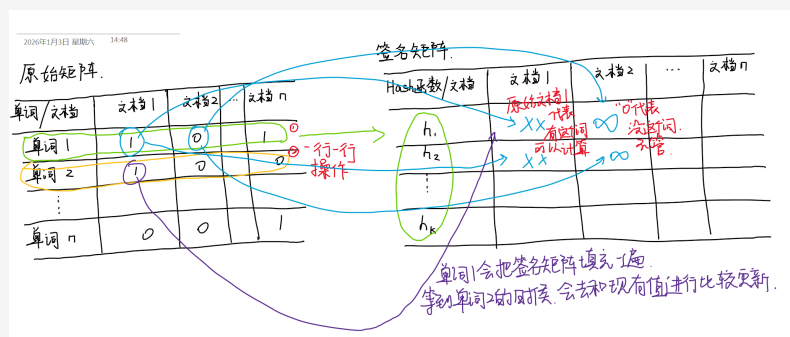


图 12.1: 原始矩阵到签名矩阵的转换示意图

LSH 的分段 (Band) 和哈希桶

MinHash 本身只是生成了签名。如果我有 100 万个文档, 即便有了签名, 两两对比还是要算 $100万^2$ 次, 依然很慢。所以 MinHash 通常配合 **LSH 的 Banding 技术和哈希桶** 使用 LSH 就像一个“过滤器”: 把不相似的文档 (绝大多数) 直接过滤掉。只有那极少数真正相似的文档, 才会因为在某一个 Band 上特征一致, 而被扔进同一个桶里, 等待最终的核实。这就把 $O(N^2)$ 的复杂度降到了近似 $O(N)$ 。

LSH 分段

核心目标: 让相似的东西更容易被分到同一个桶里

而且:

- **不相似的东西大概率不会进同一个桶**
- 只在同一个桶里做精算

做法: 我们将长长的签名向量 (对应的是签名矩阵的某一列, 比如 100 个整数), 切成 b 个段 (Band), 每段有 r 行。

- 例如: 签名长度 100, 分成 **20 个段**, 每个段包含 **5 个整数**。

哈希桶

对于我们之前分的每一个段 (Band), 我们建立一个哈希表 (Hash Table)。

- 对于文档 A, 看它的第 1 段 (包含 5 个整数, 比如 [12, 45, 1, 99, 3])。
- 将这 5 个数作为一个整体, 哈希到一个桶里 (Bucket)。
- **核心逻辑**: 只有当两个文档在**同一个段内的数值完全一模一样**, 它们才会掉进同一个桶里。

候选对生成

- 只要文档 A 和文档 B 在 **任意一个段** (无论是第 1 段还是第 20 段) 掉进了同一个桶, 我们就说它们是**“候选相似对”**。
- 如果不相似的文档, 它们在每一个段里大概率数值都不一样, 永远碰不到面。

这样做其实是充分利用了概率的想法 (**S 曲线效应**), 举个具体的数字例子: 还是假设分段 $b = 20$, 包含的整数数量 $r = 5$

- **如果两个文档很像 (有 80% 的内容一样, 即 $s = 0.8$)**:
 - 在一个段内撞桶概率: $0.8^5 \approx 0.32$
 - 在 20 个段里至少有一次撞桶的概率: $1 - (1 - 0.32)^{20} \approx 0.999$
 - **结果**: 高相似文档几乎 100% 会被捕获。
- **如果两个文档不像 ($s = 0.3$)**:
 - 在一个段内撞桶概率: $0.3^5 \approx 0.002$
 - 在 20 个段里至少有一次撞桶的概率: $1 - (1 - 0.002)^{20} \approx 0.04$
 - **结果**: 低相似文档只有 4% 的概率会被误判为候选对 (即便误判, 后续算一下真实值也就排除了)。

计算相似度

最后我们得到了这些候选对之后, 我们再去计算他们的 Jaccard 相似度, 通过签名矩阵看这两个文档

- 我们数一下这两列在**对应行**上有多少个值是相等的。
- **相等行的数量 / 总行数 $K \approx$ 这两个文档的 Jaccard 相似度。**

12.5. 精确去重与语义去重

最常见的几种去重方式包括**精确去重**、**模糊去重**、**语义去重**。其中模糊去重我们上一章已经讲了，主要利用了 MinHash+LSH 去实现。接下来我们重点关注精确去重和语义去重。

12.5.1. 精确去重

精确去重是要求完全一模一样的字符，一般会使用**加密哈希匹配**的方式去重。

假设我们要以文档为单位来去重，那么就可以对每一个文档进行 SHA-256 哈希算法，输出的哈希值存到**集合**里，接下来的文档如果会和前面的哈希值重复，那么就表示这两个文档内容重复，被丢弃。

不过我在实际的作业完成中，是以文档中的行为单位，如果有重复的行就直接去重删掉。

SHA-256 加密算法

SHA-256 (Secure Hash Algorithm 256-bit) 它的核心作用是将任意长度的输入数据，通过一系列复杂的**位运算**和**非线性函数**，转换为一个固定长度为 **256 位 (32 字节)** 的输出（摘要 digest/哈希值）。

在架构设计层面，我们需要关注以下几个核心特性：

1. **确定性**：对于相同的输入，必须永远产生相同的输出。
2. **雪崩效应**：输入数据哪怕改变 1 个 bit，输出的哈希值也应该发生巨大且不可预测的变化。
3. **单向性**：从哈希值无法通过计算的方式反推原始数据（即不可逆）。
4. **抗碰撞性**：找到两个不同的输入产生相同的哈希值，在目前的算力下应当是不可能的（虽然理论上存在，但概率极低，约为 $\frac{1}{2^{128}}$ ）。

sha-256 算法在 python 中调用非常方便，直接 hash.sha256()，例如

```
input_string = "i am pig"
# 创建 SHA-256 哈希对象
sha256_obj = hashlib.sha256()
# 更新缓冲区，必须先 encode 为 bytes
sha256_obj.update(input_string.encode(encoding))
# 返回十六进制摘要
return sha256_obj.hexdigest()
```

具体到 SHA-256 算法内部的数学操作，其实它是通过多次**移位**和**异或等位运算**，使得数据无法还原成原有的样子。SHA-256 算法还有一个更简单的替代 MD-5，也是比较原始的加密算法。

SHA-256 的缺陷

哈希算法（如 SHA-256）是确定性的。

- 如果用户 A 的密码是 “123456”，哈希值永远是 ‘8d969e...’。

- 如果用户 B 的密码也是 “123456”，哈希值也完全一样。

这带来了两个致命弱点：

1. **彩虹表攻击 (Rainbow Table Attack)**：黑客可以预先计算出所有常见密码（如 123456, password, admin）的哈希值，存入巨型数据库（彩虹表）。一旦拿到数据库泄露的哈希值，直接查表就能反查出原始密码。
2. **模式识别**：如果数据库中两个用户的哈希值相同，黑客立即知道他们使用了相同的密码。

解决方案：

Salt (盐) 是一个随机生成的字符串。在进行哈希计算之前，我们将 Salt 追加或拼接到密码上。公式从 ‘Hash(Password)’ 变为：Hash(Password + Salt)

关键设计原则：

1. **唯一性**：每个用户必须拥有独立的、唯一的盐。不可以使用全局通用的盐。
2. **随机性**：盐必须随机生成，不可预测。
3. **公开性**：盐**不需要**保密。它通常明文存储在数据库中，与哈希值放在一起。它的作用是防止查表，而不是加密。

此外，还可以加**胡椒**，Hash(Password + Salt + Pepper)，它的特点是**不在数据库里**，即使 Hash 和 Salt 泄露了，没有胡椒也不会被破解。

bcrypt/Argon2 算法

前面我们提到了为了防止黑客破解，我们可以加盐加胡椒，但如果人家就最原始的暴力破解，比如一遍遍穷举，那么也总有试出来的时候，**bcrypt/Argon2** 的思路就是把哈希计算的速度变慢**变慢**，这样以来，你穷举的成本被大大提高，就可以间接保护密码不被泄露了。

12.5.2. 语义去重

MinHash+LSH 的模糊去重的原理是基于**字面结构**的相似。它关注的是**字符或单词的重叠率**。而**语义去重**：基于**潜在含义的相似**。它关注的是**向量空间中的几何距离**。就是 Transformer 转为稠密向量那一套，语义去重的核心是将离散的文本转化为连续的稠密向量，利用**向量的方向一致性来判断语义相似度**。

1. 向量化

使用预训练的语义模型（如**BERT, RoBERTa, BGE, text-embedding-ada-002**）将每条文本映射为一个固定维度的向量（例如 768 维或 1536 维）。

本质：将语义信息压缩到向量数值中。

2. **相似度矩阵计算** 计算向量之间的**余弦相似度**。对于 N 条数据，理论上需要计算 $N \times N$ 的矩阵。

- 比如：

● $V_A = [0.85, 0.12]$

- $V_B = [0.84, 0.13]$
- $\text{Cosine}(V_A, V_B) = \frac{V_A \cdot V_B}{\|V_A\| \|V_B\|} \approx 0.99$ 方向非常接近。
- 优化：由于 $O(N^2)$ 复杂度过高，通常配合 **向量检索库 (FAISS)** 或 **聚类算法 (K-Means/DBSCAN)** 来缩小对比范围，仅计算同一个**簇 (Cluster)**内的数据。
 - 两种方式其实比较接近，都是希望找最接近的那一批再精细化比较，避开粗颗粒上就离得比较远的数据。
 - FAISS 的 IVF (**倒排索引**) 会预先训练出一组**虚拟的中心点**，因此，有新的向量之后会先去计算距离哪个中心点比较近，之后就只去和这个中心点附近单元的数据比较，避免了遍历所有的数据。它的特点是它是动态的，**数据是源源不断进来的**
 - 聚类的算法比较容易，就是一次性把所有的数据**分簇**，只需要在同一簇内的向量进行相似度比较就行了，不同簇的相似度大概率不如同一簇内向量的相似度，不用比了。
- 3. **阈值判定** 设定一个较高的语义相似度阈值（通常 ≥ 0.90 或 ≥ 0.95 ）。
 - 若 $\text{Similarity}(A, B) > \text{Threshold}$ ，则判定 A 与 B 语义重复。

4. 筛选

简单例子

- **文本 A**：“光速是多少？”
- **文本 B**：“真空中的光传播速度数值。”

如果采用模糊去重，那么这两句话因为重复的文本很少，就会被认为是不相似。但实际上他们的语义及其相似。利用语义向量来计算余项相似度的话就能看到他们的方向及其接近。